Learn Japanese with Manga!

Building a Multimodal Search Engine with Gemini Embedding 2

Jeff Y. Hung (@x0mg) · PyCon TW 2026

About me

Jeff · @x0mg

- 🐍 Python for ~9 years — data engineering, ML, automation, security
- 🔁 PyCon TW **2019 attendee → 2026 speaker**

github.com/jeff14994 · jeff14994.github.io

Why manga?

- Manga + anime = **the same story in two media**: read it, then hear it
- But how do you find that scene again?
  - filenames: `001.webp`, `S1E01.mp4`
  - tags & episode numbers don't capture **meaning**
- I wanted: **read a panel → find the scene → hear it in Japanese**

Manga-Lingo in action

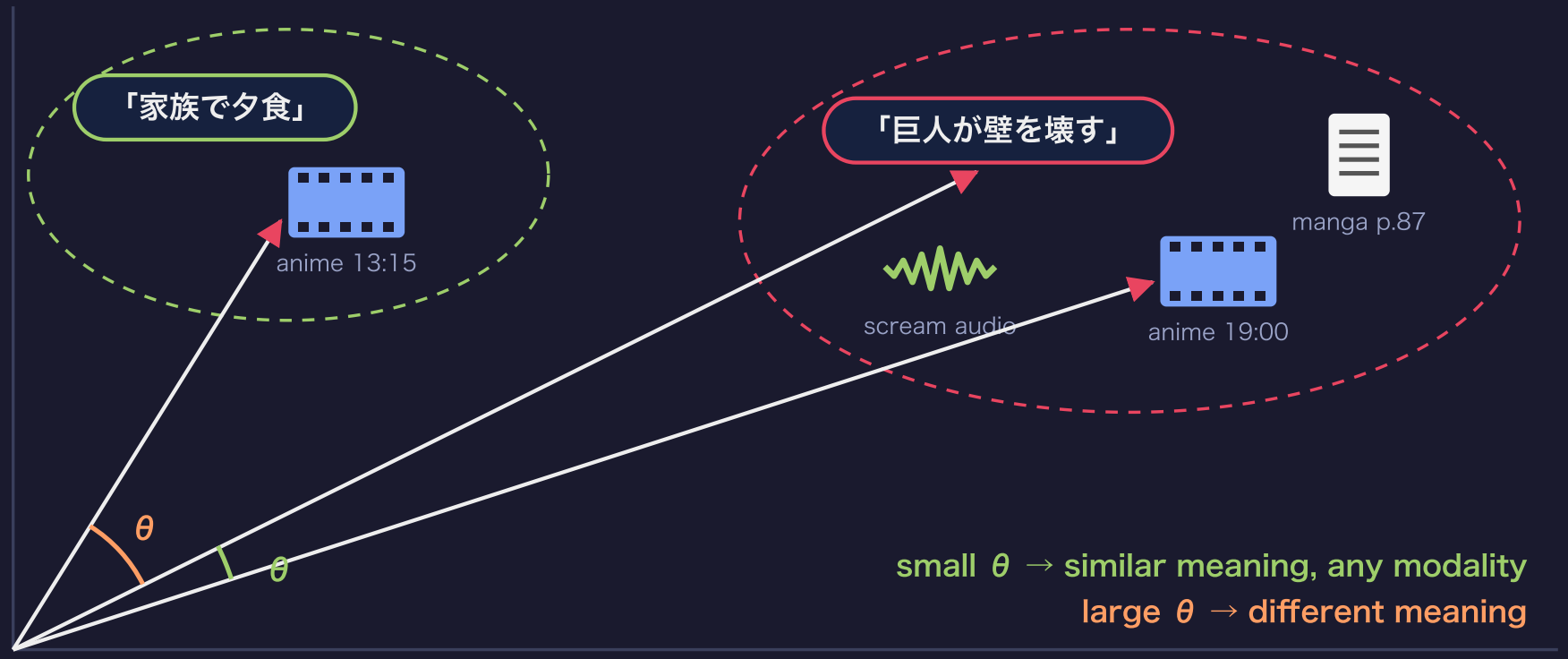
Left: a manga page · **Right:** the matched anime scene + timestamped subtitles

(live demo)

Agenda

- 🧭 Embeddings in two minutes
- 1️⃣ **Architecture** — one vector space, PostgreSQL + pgvector
- 2️⃣ **Batch ingestion** — asyncio, semaphores, 15 s video chunks
- 3️⃣ **Does 3072 beat 768?** — 44 hand-labeled queries
- 4️⃣ **Live demo** — text, image & audio search · study · compare
- 5️⃣ **Bonus** — from search to studying Japanese
- 🙋 Q&A (5 min)

Embeddings in one slide



Content → **vector** · similar meaning → **small angle** · **one space** for every modality

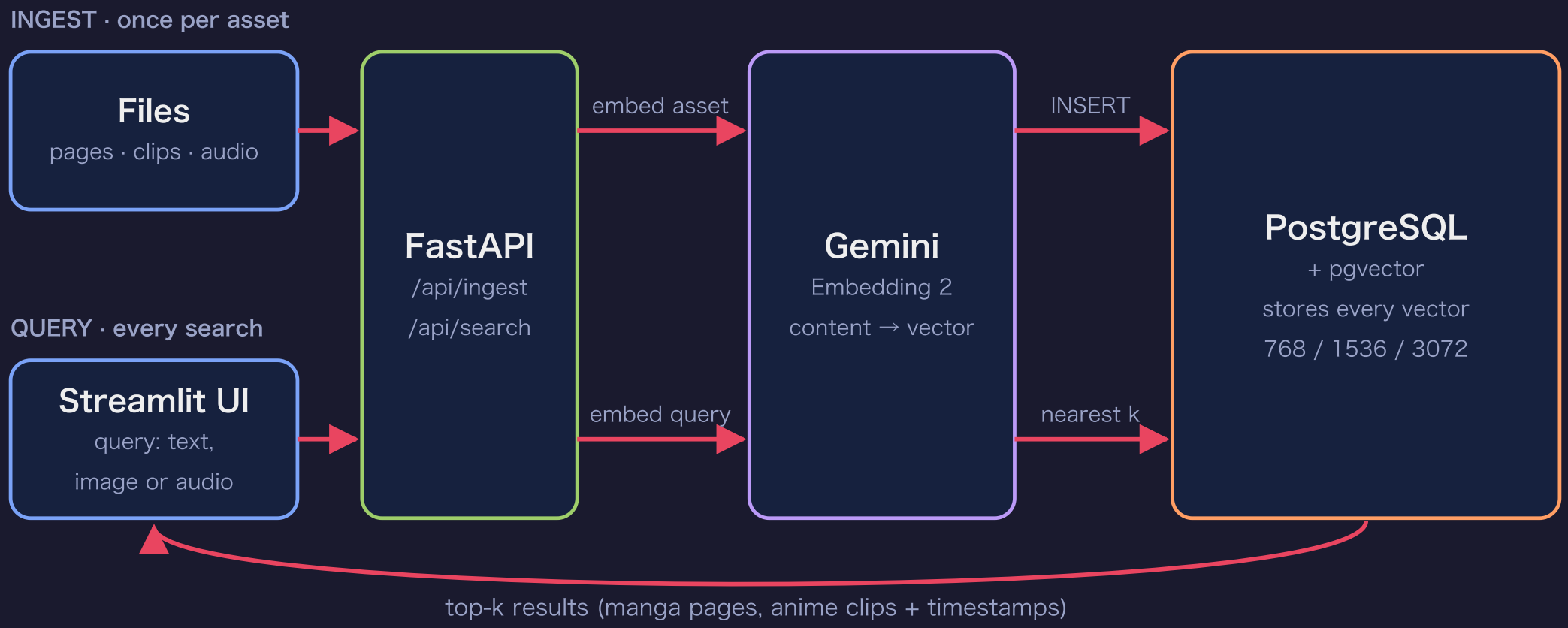
Gemini Embedding 2

Feature	
Inputs	text, image, audio, video, PDF
Output	3072 dims by default
Matryoshka (MRL)	truncate to **1536** or **768** with little loss
Video limit	short clips → we cut episodes into **15 s chunks**

```
payload = {
    "content": {"parts": [{"inlineData": {"mimeType": "image/jpeg", "data": b64}}]},
    "outputDimensionality": 768,     # 768 / 1536 / 3072
}
requests.post(f"{VERTEX}/models/gemini-embedding-2-preview:embedContent", json=payload)
```

1 Architecture

System overview



Vectors are computed **once** and **stored** · a search embeds **only the query**

Schema: one column per dimension

```sql
CREATE TABLE embeddings (
    asset_id        INT REFERENCES assets(id) ON DELETE CASCADE,
    modality        VARCHAR(10),              -- image / video / audio
    embedding_dim   INT,                      -- 768 / 1536 / 3072
    embedding_768   vector(768),
    embedding_1536  vector(1536),
    embedding_3072  vector(3072)
);
```

- An **index** needs a fixed dimension (`vector(768)`) → one column per size
- One row per (asset, dimension) → compare dimensions **on the same assets**
- Adding a dimension later = insert rows, **don't re-ingest** (`reembed.py`)

pgvector gotcha: the 2000-dimension wall

```
ERROR: column cannot have more than 2000 dimensions for ivfflat index
```

- Index entries must fit in one **8 KB page**: 3072 × 4 bytes = 12 KB ❌
- Fix: index a `halfvec` cast (2 bytes/dim, up to 4000 dims)

```
CREATE INDEX idx_emb_3072 ON embeddings
    USING hnsw ((embedding_3072::halfvec(3072)) halfvec_cosine_ops);

-- queries must use the same cast to hit the index
ORDER BY embedding_3072::halfvec(3072) <=> %s::halfvec(3072)
```

> At 436 rows per dimension Postgres **correctly ignores** the index — a sort is cheaper.

pgvector gotcha 2: IVFFlat is approximate

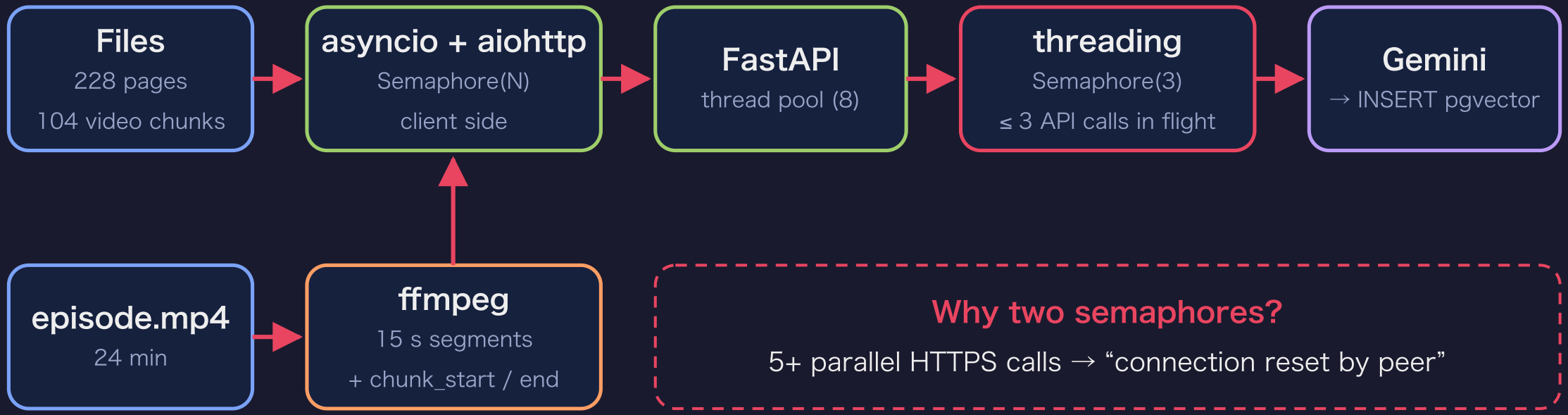
IVFFlat splits vectors into clusters (`lists=10`).
Default `probes=1` → searches **1 of 10** → can miss better matches in the other 9.

```
SET ivfflat.probes = 10;  -- search all 10 clusters → exact results
```

> At 436 rows & probes=lists, the cost is ~1–2 ms. At 100 K+ you'd lower probes and accept approximate.

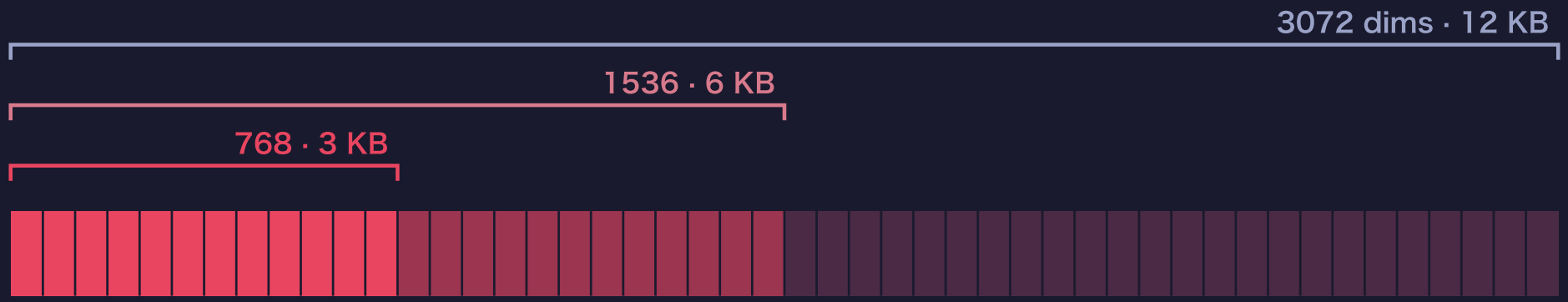
2 Batch ingestion

asyncio + aiohttp, but throttled



228 pages · 6 min · **104** anime chunks · 3 min · re-embed **664** vectors · 12 min

3 Does 3072 beat 768?

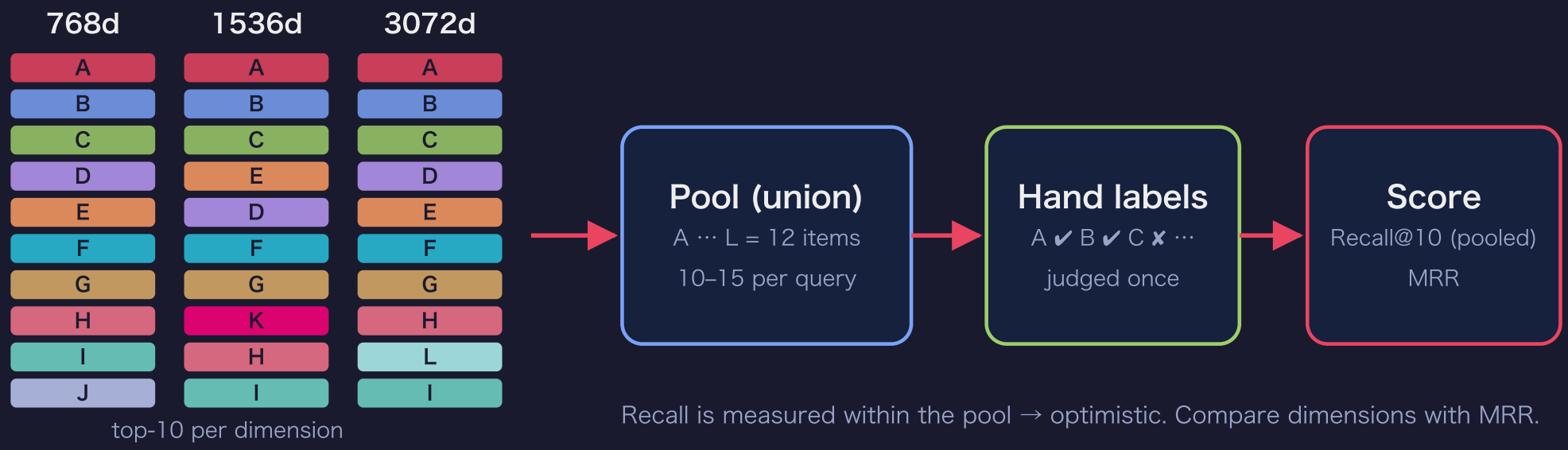


One embedding, nested prefixes: a smaller vector = the first N numbers

`outputDimensionality = 768 | 1536 | 3072`

How I measured

44 queries · 4 types: text→image, text→video, image→video, audio→video



Recall is measured within the pool → optimistic. Compare dimensions with MRR.

Results

All **44** queries = 24 normal + 10 hard + 10 audio

Dim	Recall@10	MRR	Search p50	Bytes / vector
768	**0.956**	**0.883**	**5.3 ms**	**3 KB**
1536	0.928	0.878	6.8 ms	6 KB
3072	0.938	0.880	12.1 ms	12 KB

- Quality: **within noise** across dimensions
- Cost: 3072 = **4× storage**, **2.3× search latency**
- Query embedding ≈ 1.2 s at every size (network-bound)

Harder queries: what actually fails

MRR by query group (1.0 = the first result is always correct)

MRR	768	1536	3072
Normal (24)	0.95	0.97	0.97
Hard (10)	**0.78**	**0.71**	**0.71**
Audio (10)	0.83	0.83	0.83
All (44) ← previous slide	0.883	0.878	0.880

- ✅ Text **inside the panel** 「瑪利亞之牆被突破了」 → #1
- ✅ JP query → CN page 「巨人が壁を壊す」 → #1
- ❌ Abstract emotion 「家族を失う悲しみ」 → #5–6
- ❌ Small details 「城牆上的大砲」 → #3–6

> Failures follow the **query type**, not the vector size.

Audio → video: embed the audio track

Clip (6–10 s)	vs video	vs audio track	📱 phone · audio track
Opening song	❌	**#1**	**#1**
Dialogue 「立体機動に移れ」	#2	**#1**	**#1**
Mother scene (screams)	#1	**#1**	**#1**
「エレン…どうして泣いてるの」	#1	**#1**	**#1**
Colossal Titan appears	#1	#3	❌

- A video embedding is **dominated by the visuals** → music-heavy scenes can't be found by sound
- Embed each chunk's **audio track** too: audio MRR **0.66** → **0.83**

Decision table

If you need…	Choose
Semantic search on a small/medium collection	**768**
Storage or latency matters	**768** (¼ the bytes)
Hard queries failing	better queries / data, **not** 3072
> ~2000 dims with an index on vanilla pgvector	`halfvec`
Search by audio	embed the **audio track**, not just the video

4 Live demo

Demo 1・「巨人が壁を壊す」 → manga page

Japanese query → the Wall Maria page of the Chinese edition at **#1**

(screenshot removed for public release)

Demo 2 · Manga page → anime clip

Manga page → matched anime clip with timestamped subtitles

(screenshot removed for public release)

Demo 3 · Study: read and listen

Read the page bubble by bubble; ▶️ plays the anime from each line

(screenshot removed for public release)

Demo 4 · Phone recording → anime scene

Phone-quality audio clip → 22:45 「やめろ！」 at **#1**

(screenshot removed for public release)

Demo 5・「醜い巨人」 at 768 / 1536 / 3072

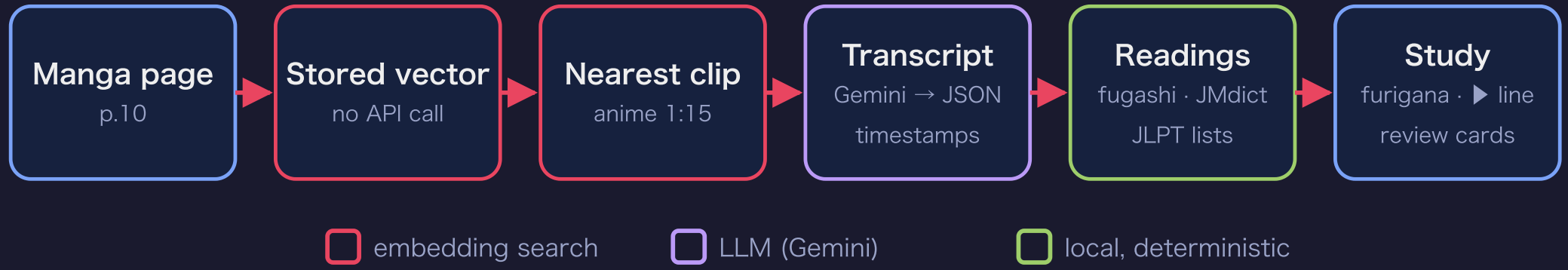
Same top result at 768, 1536 and 3072

(screenshot removed for public release)

5 Bonus

From search to studying Japanese

From search to studying



Audio says	gemini-2.5-flash	gemini-3-flash-preview
りったいきどうに **うつれ**	立体機動に **映れ** ❌	立体機動に **移れ** ✅

Takeaways

1. **One embedding space** makes page ↔️ clip ↔️ audio search a few lines of SQL
2. **Measure before you pay**: 768 matched 3072 at ¼ the storage
3. Failures come from **query type** and **what you embed**, not dimension
4. **Vanilla PostgreSQL + pgvector** is enough — mind the 2000-dim index limit
5. Use LLMs where they shine (OCR, transcription); keep **readings deterministic**

Thank you!



github.com/jeff14994/manga-lingo

code · eval scripts · 44 labeled queries

Questions?